# Reviewer Pack — Minimal Representation of p-Adic Logarithmic Capacity and Co…

Entry 781e89a99b43 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-10 10:44:07 UTC

## Minimal Representation of p-Adic Logarithmic Capacity and Communication Complexity Rank Variance

**Entry ID**: `781e89a99b43` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-10 10:44:07 UTC

### 1. Conjecture

**Field A** (mathematical branch): p-Adic Number Theory **Field B** (complexity object): Communication Complexity - Rank Variance

**Statement**:

> For all boolean functions f with n inputs, the p-adic logarithmic capacity (log-capacity) of the associated hyperplane arrangement is asymptotically equal to the communication complexity rank variance of f, i.e., log-cap(f) = Θ(rank_variance(f)).

**Rationale (proposer's reasoning)**:

> The use of p-adic number theory in communication complexity could reveal new connections between arithmetic structures and combinatorial problems. Log-capacity measures the arithmetical complexity of a set of points in projective space, while rank variance captures the complexity of evaluating functions on these sets. This conjecture suggests that there is an inherent relationship between these two measures that can be exploited for complexity lower bounds.

**Taxonomy category**: `p_adic_log_capacity` (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `1c158257f23f1667`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

> The conjecture is supported if the mean correlation coefficient between log-capacity and communication complexity rank variance across all tested boolean functions f with n inputs is ≥ 0.8, and the standard deviation of this correlation does not exceed 0.1.

### 3. Barrier filter (F1)

**Final**: `PASS`

| Barrier | Verdict | Confidence | $LLM_1$ | $LLM_2$ |
|---|---|---|---|---|
| RELATIVIZATION | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| NATURAL_PROOFS | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| ALGEBRIZATION | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| KARP_LIPTON | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |

## 4. Novelty audit

**Verdict**: `ADJACENT_OK` against 3 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (2): - `"p-adic logarithmic capacity" AND "communication complexity rank variance"` - `"hyperplane arrangement" IN p-Adic Number Theory AND communication complexity"`, `"asymptotic equality" IN (log-capacity) AND (rank variance)`

**Top relevant hits considered**: - [http://arxiv.org/abs/0707.3682v2] On the p-adic Beilinson conjecture for number fields - [http://arxiv.org/abs/math-ph/0512018v2] On Phase Transitions for $P$-Adic Potts Model with Competing Interactions on a Cayley Tree - [http://arxiv.org/abs/2408.00810v3] p-adic Equiangular Lines and p-adic van Lint-Seidel Relative Bound

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=-9, elapsed=120.0s

### 5.1 Generated Python source

```python
import random
import math
from fractions import Fraction
from itertools import combinations, product

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def hyperplane_arrangement(f):
        n = len(f)
        arrangement = []
        for i in range(n):
            if f[i] == 1:
                arrangement.append([i])
        return arrangement

    def log_capacitance(arrangement, p=2):
        n = len(arrangement)
        if n == 0:
            return 0
        sum_log = 0
        for subset in combinations(range(n), n-1):
            product_val = 1
            for i in subset:
                product_val *= (i + 1)
            sum_log += math.log(product_val, p)
        return sum_log / n

    def communication_complexity_rank_variance(f):
```

```
    n = len(f)
    rank_var = 0
    for k in range(1, n+1):
        subsets = list(combinations(range(n), k))
        for subset in subsets:
            count_ones = sum(f[i] for i in subset)
            if count_ones == 0 or count_ones == k:
                continue
            rank_var += (count_ones - k/2)**2 / k
    return rank_var

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    instances_tested = 0
    total_log_cap = 0
    total_rank_var = 0

    for _ in range(30):
        f = [random.randint(0, 1) for _ in range(n)]
        arrangement = hyperplane_arrangement(f)
        log_cap = log_capacitance(arrangement)
        rank_var = communication_complexity_rank_variance(f)

        if log_cap <= 0 or rank_var <= 0:
            continue

        instances_tested += 1
        total_log_cap += log_cap
        total_rank_var += rank_var

    if instances_tested == 0:
        continue

    mean_log_cap = total_log_cap / instances_tested
    mean_rank_var = total_rank_var / instances_tested
    correlation_coefficient = (instances_tested * mean_log_cap * mean_rank_var -
                               total_log_cap * total_rank_var) / (
                                   math.sqrt(instances_tested *
                                             (total_log_cap**2 - instances_tested * mean_log_c
                                   math.sqrt(instances_tested *
                                             (total_rank_var**2 - instances_tested * mean_rank

    results.append({
        "n": n,
        "mean_log_cap": mean_log_cap,
        "mean_rank_var": mean_rank_var,
        "correlation_coefficient": correlation_coefficient
    })

if not results:
    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": None,
        "instances_tested": 0,
```

```
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "No valid instances found"
        }

    mean_corr_coeff = sum(result["correlation_coefficient"] for result in results) / len(results)
    std_corr_coeff = math.sqrt(sum((result["correlation_coefficient"] - mean_corr_coeff)**2 for re

    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": mean_corr_coeff,
        "instances_tested": sum(result["instances_tested"] for result in results),
        "n_max": max(result["n"] for result in results),
        "conjecture_holds": mean_corr_coeff >= 0.8 and std_corr_coeff <= 0.1,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2**i - 1 for i in range(5, 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...{result}...}}")
        results.append(result)

    mean_corr_coeff = sum(result["metric_value"] for result in results) / len(results)
    std_corr_coeff = math.sqrt(sum((result["metric_value"] - mean_corr_coeff)**2 for result in res
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_corr_coeff} std={std_corr_coeff} support_fraction={su
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_corr_coeff} std={std_corr_coeff} support_fraction={su
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"Correlation coefficient did not meet threshold\
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
(empty)
```

## 8. Critic adversarial review

**Critic verdict**: CHALLENGE

**Critic reasoning**:

> skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict**: `INCONCLUSIVE`

**Reasoning**:

> The test crashed before producing data, which means that the mean correlation coefficient and standard deviation of correlation could not be calculated. | next: Re-run the test to ensure it completes successfully and provides the necessary data for calculating the mean correlation coefficient and standard deviation of correlation.

## 11. Audit log (LLM calls)

**Total LLM calls**: 9

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | glm4:latest | 0 | 0 | 15007 |
| 2 | preregistration | ollama_remote | glm4:latest | 0 | 0 | 12130 |
| 3 | novelty | ollama_remote | glm4:latest | 0 | 0 | 8733 |
| 4 | novelty | ollama_remote | glm4:latest | 0 | 0 | 9438 |
| 5 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 43695 |
| 6 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 15484 |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 38296 |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 16472 |
| 9 | judge | ollama_remote | glm4:latest | 0 | 0 | 80975 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 240230 ms total latency. Provider mix: {'ollama_remote': 9}

*(full prompt+response transcripts available in `research/audit/781e89a99b43.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/781e89a99b43.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/781e89a99b43.tar.gz` (if generated)